

Phase I

(Setup)

External libraries:-

python-can (import can)
pyisotp (import isotp)
udsoncan (import udsoncan)
openpyxl (import openpyxl)

In-build libraries:-

time (import time)
threading (import threading)

T
o
o
l

c
o
d
e

Define:
[load_requests_from_excel()
[decode_did_value(did, data)]



Call: [load_requests_from_excel()]



Call: [can.Bus()]
Config: interface='udp_multicast'
Params: channel='-', port=-



Call: [isotp_address()]
Setup: rxid=0x7E8, txid=0x7E0



Call: [isotp.CanStack()]
Argument: bus=bus, address=addr
Params: stmin=0, blocksize=8,
tx_padding=0x00
(standard)



Setup Application Timeout Options
Call: [default_client_config.copy()]
Config: custom_config['p2_timeout'] = 60.0
Config: custom_config['p2_star_timeout'] = 60.0

E
C
U

c
o
d
e

Define:
[load_ev_state()]
[state_polling_thread()]
[process_uds_request(payload)]



Call: [threading.Thread()]
Argument: target=[state_polling_thread]
Argument: daemon=True



Call: [load_ev_state()]
Params: time.sleep(2.0)
(refresh period)



Call: [can.Bus()]
Config: interface='udp_multicast'
Params: channel='-', port=-



Call: [isotp_address()]
Setup: rxid=0x7E0, txid=0x7E8



Call: [isotp.CanStack()]
Argument: bus=bus, address=addr
Params: stmin=0, blocksize=8,
tx_padding=0x00
(standard)

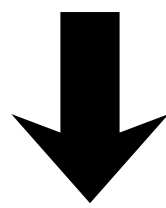
PHASE II

(User request generation)
(Tool code)

Event Listener Active (application in idle state)

call : `.get()/.text() & app.exec_()`

eg. output : 22 01 05

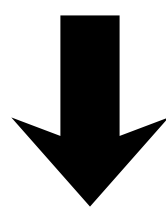


Build Targeted UDS Request Frame Array

Call: `.replace(" ", "")`

Call: `bytes.fromhex(sanitized_input)`

eg output : `b'\x22\x01\x05'`



Execution Call: `[client.send_request()]`

Internal Network Sub-Call: `[stack.send()]`

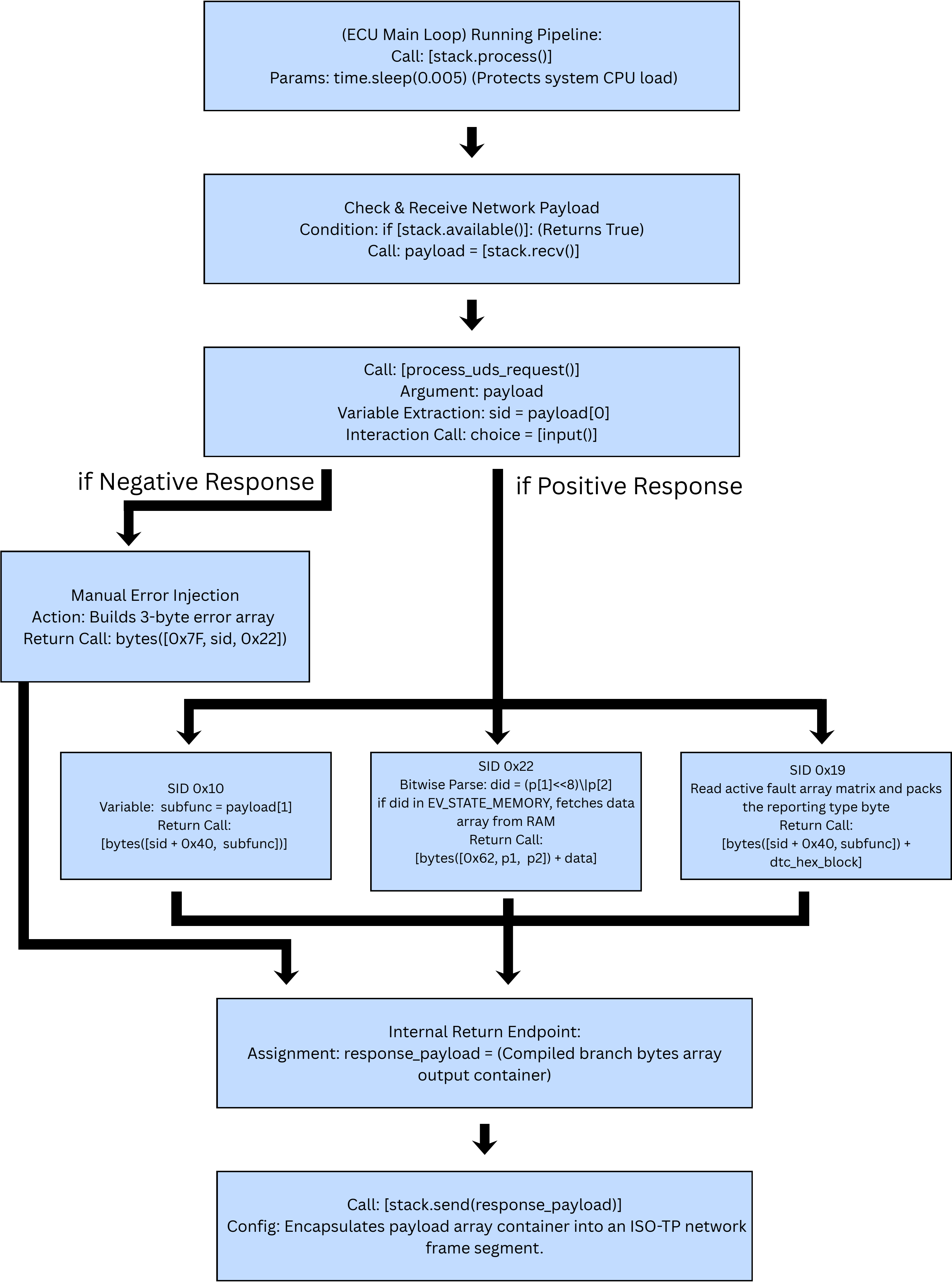


TX PIPELINE: ISO-TP
Slices Multi-frame
Data Packets

Phase III

(The ECU Interpretation)

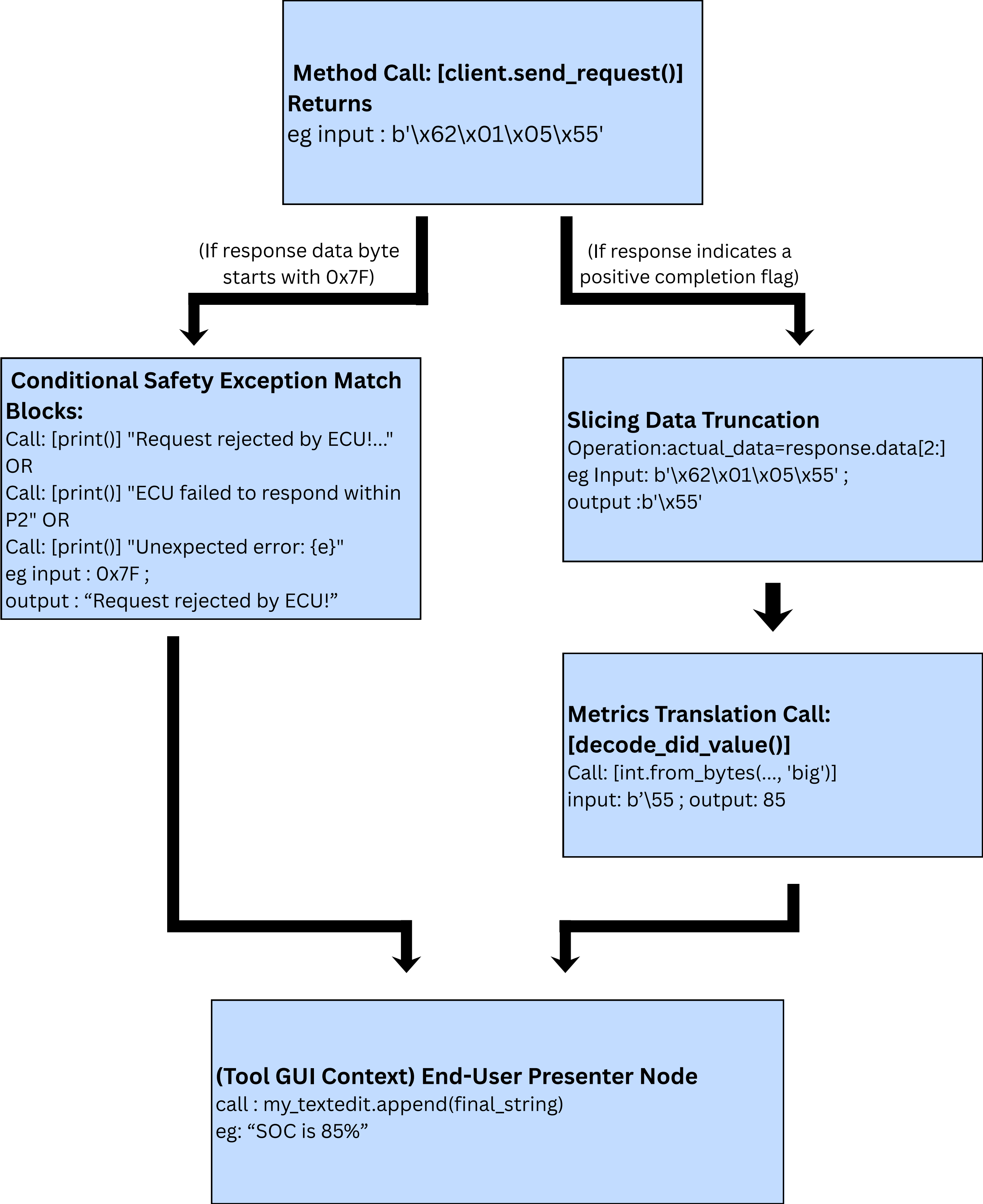
(ECU code)



PHASE IV

(Decoding and Showing the result)

(Tool code)



Software architecture of open source EV diagnostic engine

